

Virtual Mouse

Hand Gesture Control Pipeline via MediaPipe & OpenCV

The Pipeline Architecture



1. Capture

Read from webcam using OpenCV, flip for natural mirroring, and convert to RGB format.



2. Detect

MediaPipe processes the frame, extracting 21 normalized coordinate landmarks per hand.



3. Logic

Calculate distances and states to detect pinches, scrolls, and cursor targeting.



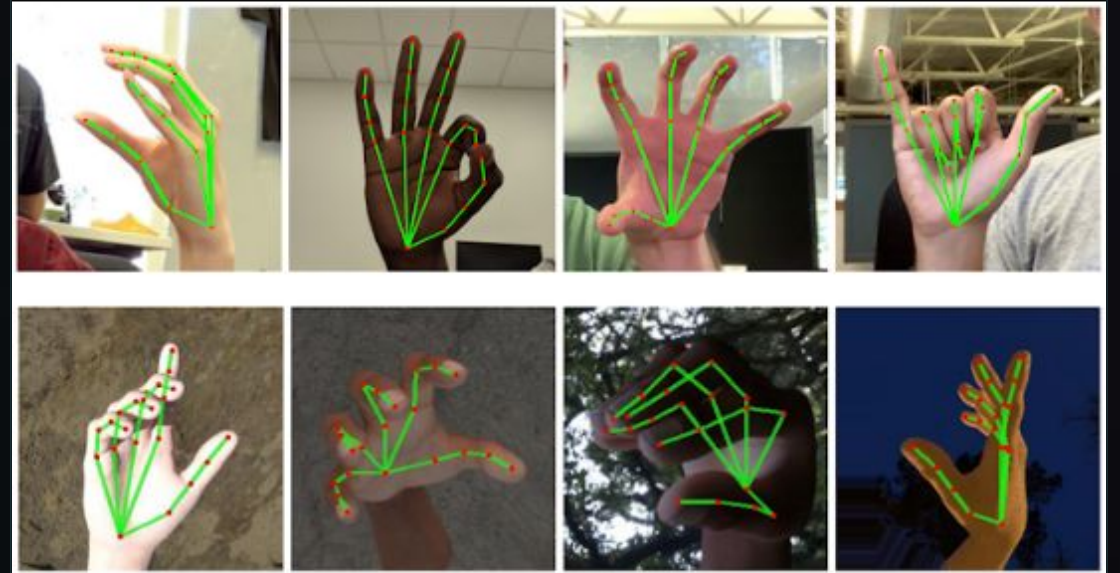
4. Action

PyAutoGUI executes the final mapped screen coordinate movements and click events.

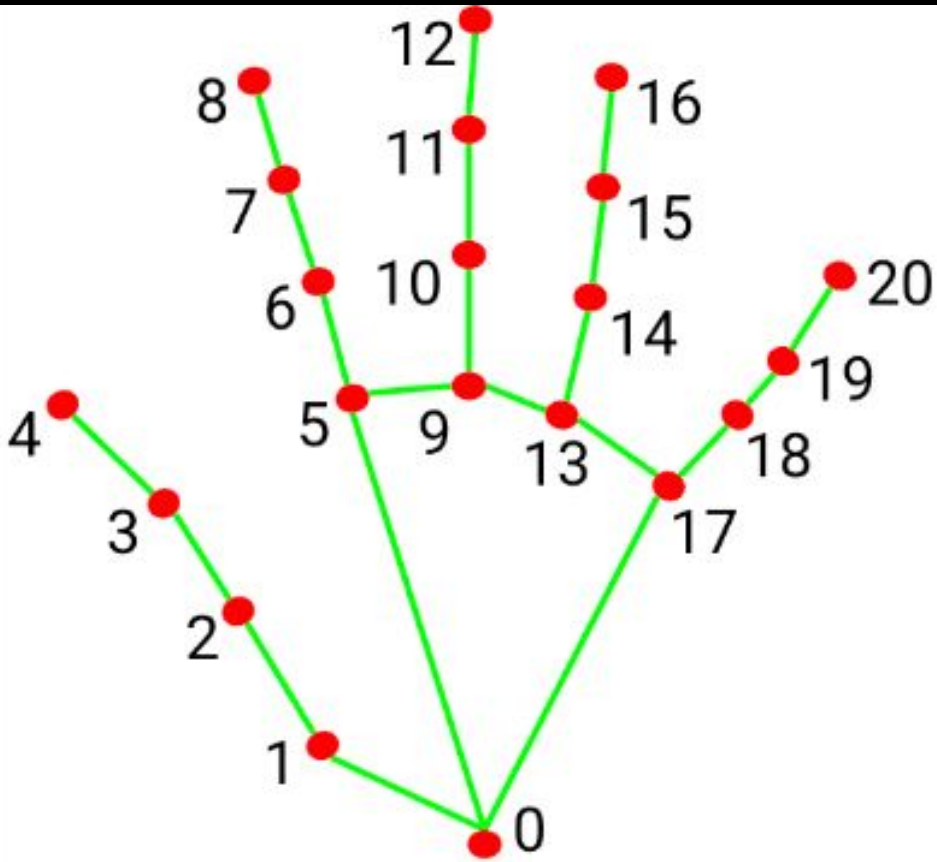
MediaPipe Landmarks

The core tracking relies on a 21-point 3D topology map provided by MediaPipe's pre-trained machine learning model.

- ▶ **Filtering Noise:** Configured for `max_num_hands=1` and `min_detection_confidence=0.7` to drop shaky frames.
- ▶ **Key Tracking Points:**
Thumb Tip (4), Index Tip (8), Middle Tip (12)
- ▶ **Reference Joints:** We use the lower PIP joints (e.g., 6, 10) to determine if a finger is logically "up" or "down".



MediaPipe **Hand** Detection



- 0. WRIST
- 1. THUMB_CMC
- 2. THUMB_MCP
- 3. THUMB_IP
- 4. THUMB_TIP
- 5. INDEX_FINGER_MCP
- 6. INDEX_FINGER_PIP
- 7. INDEX_FINGER_DIP
- 8. INDEX_FINGER_TIP
- 9. MIDDLE_FINGER_MCP
- 10. MIDDLE_FINGER_PIP

- 11. MIDDLE_FINGER_DIP
- 12. MIDDLE_FINGER_TIP
- 13. RING_FINGER_MCP
- 14. RING_FINGER_PIP
- 15. RING_FINGER_DIP
- 16. RING_FINGER_TIP
- 17. PINKY_MCP
- 18. PINKY_PIP
- 19. PINKY_DIP
- 20. PINKY_TIP

(image from official mediapipe documents)

Finger **State** Detection

Image Coordinate System

In standard computer vision (OpenCV), the origin (0,0) is at the top-left of the image. This means the Y-axis increases as you move down.

Therefore, a finger is computationally "extended" (up) if its tip's Y-coordinate is **less than** the Y-coordinate of its lower joint.

The Logic Array

We generate a binary state array evaluating ``tip.y < joint.y`` for the index, middle, ring, and pinky fingers:

`[1, 1, 0, 0]` = Index & Middle up.

`sum(fingers) == 4` is the trigger condition utilized later to enter "Scroll Mode".

The Hysteresis Fix

Eliminating Jitter and Phantom Clicks in Gesture Recognition

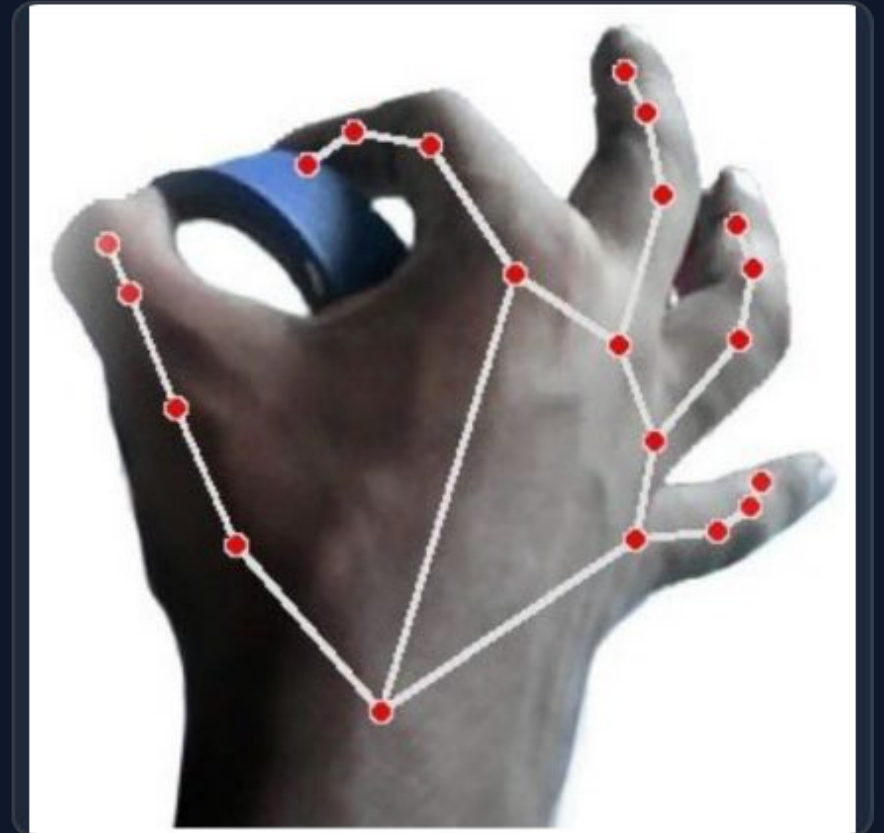
The Math of a Pinch

Before we detect a click, we must measure the distance between the **Thumb Tip (4)** and the **Index Tip (8)**.

We use the Euclidean distance formula, implemented via Python's `math.hypot()`.

$$d = (x_2 - x_1)^2 + (y_2 - y_1)^2$$

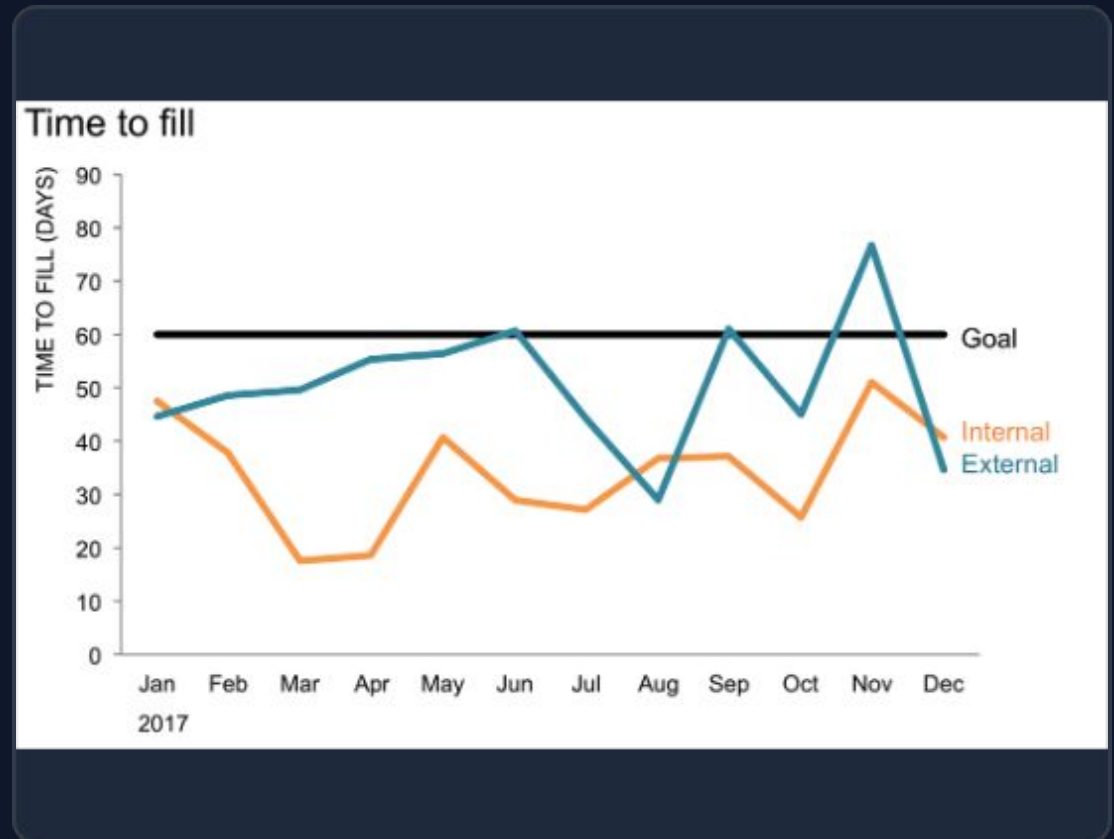
When this distance `d` drops to a certain value, the fingers are physically touching, representing a "click".



The Single Threshold Problem

Initially, one might use a single threshold: If $d < 0.060$, then CLICK.

- ▶ **The Flaw:** Human hands naturally tremble.
Webcams have pixel noise.
- ▶ **The Result:** If your fingers hover exactly near the 0.060 boundary, the distance reading will rapidly fluctuate (e.g., 0.061, 0.059, 0.061).
- ▶ **The Consequence:** The system rapidly fires ON and OFF, causing dozens of unintended "phantom clicks" per second.



The Dual Thresholds: **Hysteresis**

1. The Enter State

CLICK_ENTER = 0.055

To register a brand new click, the distance must drop **below 0.055**.

This requires a deliberate, tight physical pinch.
Once crossed, the system locks into the
`freeze_cursor = True` (Clicked) state.

2. The Exit State

CLICK_EXIT = 0.070

To release the click, the distance must expand **above 0.070**.

This requires a deliberate physical separation of the fingers. The system will not release the click until this wide boundary is crossed.

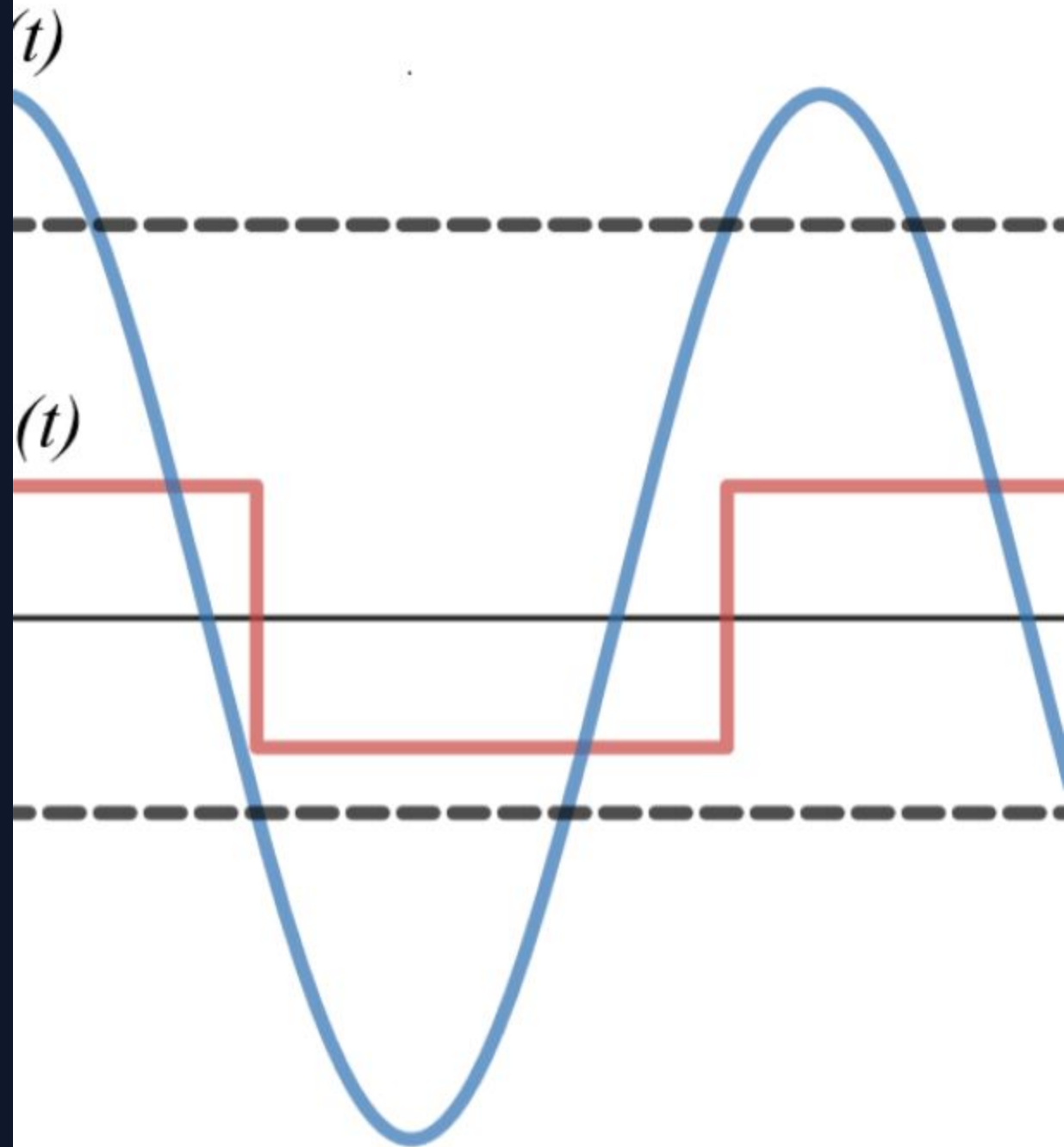
The Deadband

Immunity to Noise

The gap between 0.055 and 0.070 creates a buffer zone called a **Deadband**.

If the distance d fluctuates inside this zone (e.g., 0.062), **the state does not change**.

If it was clicked, it stays clicked. If it was released, it stays released. The micro-tremors are absorbed entirely by this gap.

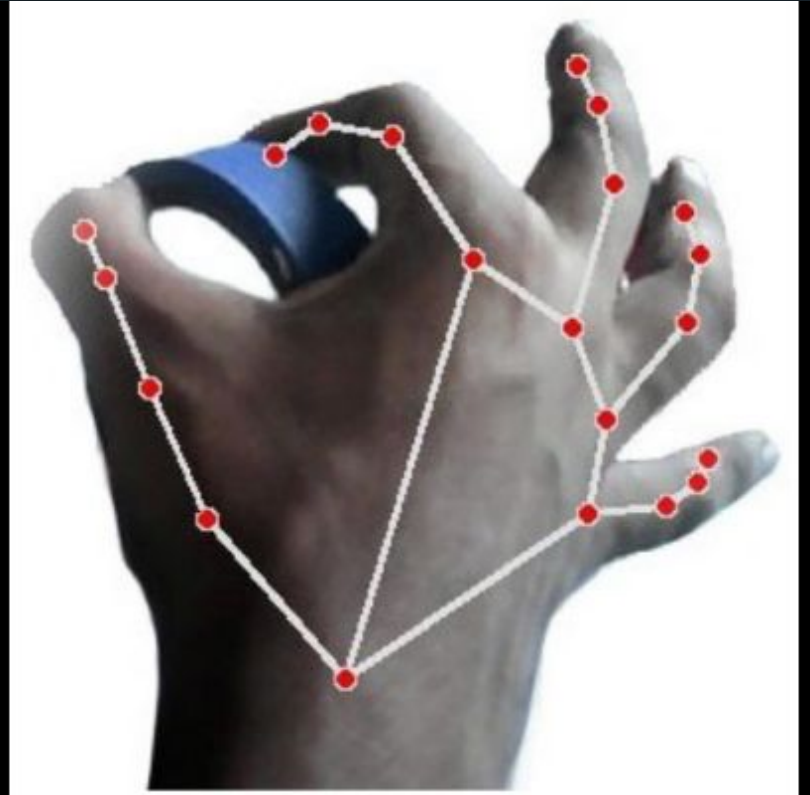


Pinch & Click Detection

A "click" is registered using a pinch gesture—measuring the Euclidean distance between the thumb tip and the index tip.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- ▶ Utilizes Python's `math.hypot()` for fast calculation.
- ▶ Double-click timing logic verifies if consecutive pinches occur within a 0.4-second threshold window.



The Hysteresis Fix

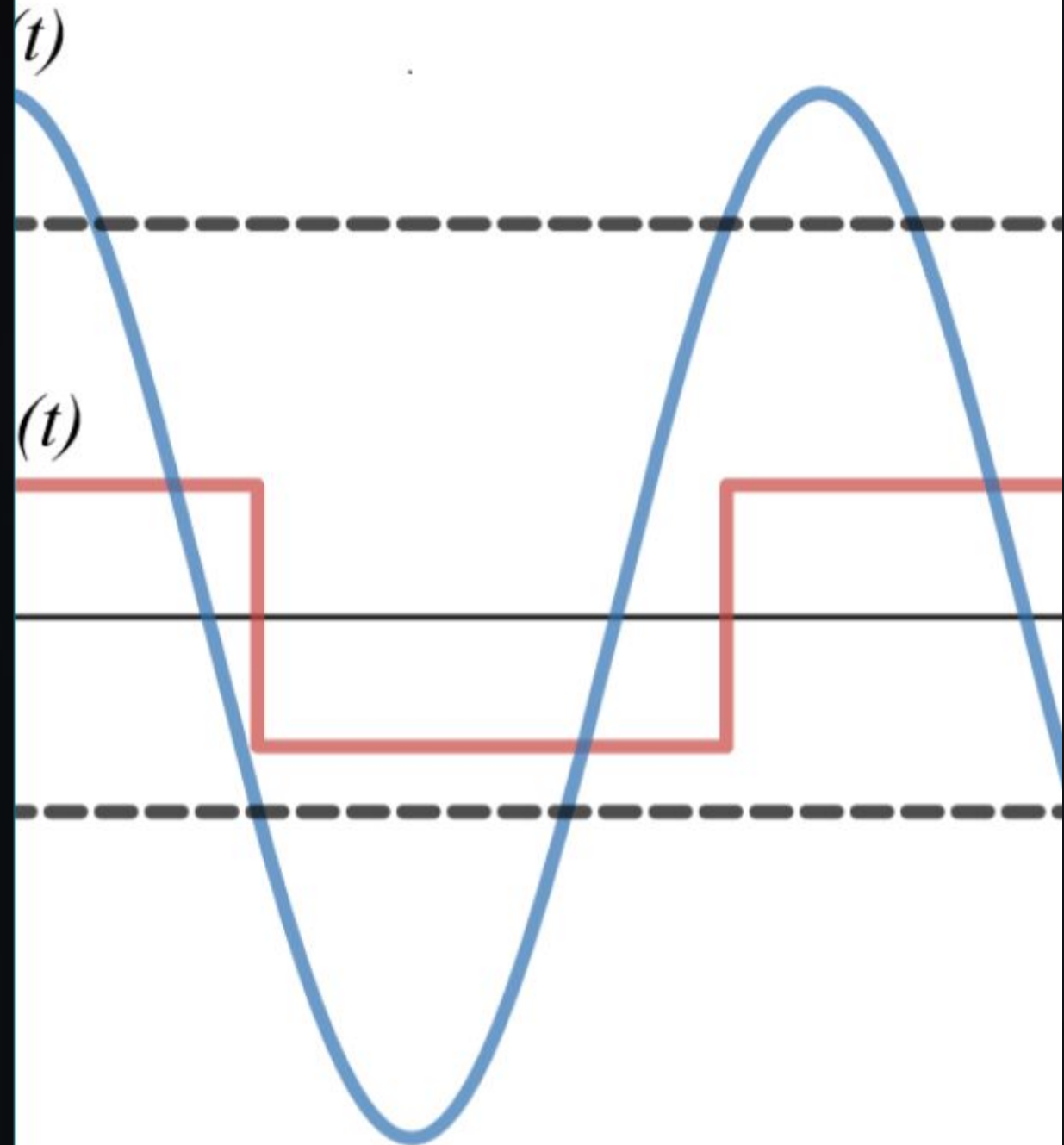
Preventing Jitter

A single threshold for clicking causes erratic "phantom clicks" when hand micro-tremors hover at the boundary.

Solution: We implement a control system concept called Hysteresis.

- **Enter State:** $\text{dist} < 0.055$
- **Exit State:** $\text{dist} > 0.070$

This physical gap creates a stable binary state that entirely neutralizes borderline noise.



Rock-Solid *Stability*

By implementing Hysteresis, we transform a noisy, unstable analog signal into a confident, flawless digital state.

Image Sources



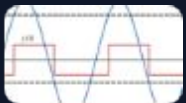
https://www.mdpi.com/technologies/technologies-14-00334/article_deploy/html/images/technologies-14-00334-g008-550.jpg

Source: www.mdpi.com



<https://images.squarespace-cdn.com/content/v1/55b6a6dce4b089e11621d3ed/1525827943786-YDFT4MZADG6860CXG3Z8/Dotted+Lines+1.p>

Source: www.storytellingwithdata.com



https://www.typhoon-hil.com/documentation/typhoon-hil-software-manual/Images/hysteresis_function_03.png

Source: www.typhoon-hil.com

Cursor Kinetics

Mathematical Mapping & Signal Smoothing in Virtual Environments

Cursor Kinetics

1. The Active Zone Remap

Reaching the physical edge of the webcam frame is unnatural. We define an inner "active zone" (reduced by `frame_reduction`) and use `np.interp` to stretch it.

Moving a hand slightly within the center of the camera easily covers 100% of the screen resolution.

2. Exponential Moving Average

Raw coordinates jump pixel-to-pixel. We dampen this noise using a blended smoothing formula.

$$P_{\text{new}} = P_{\text{old}} + \frac{P_{\text{target}} - P_{\text{old}}}{k}$$

Where `k` is our smoothing factor (5). The higher the factor, the smoother (but slightly slower) the cursor.

The Reachability Problem

1. The Hardware Limit

MediaPipe returns normalized coordinates from 0.0 to 1.0 relative to the webcam feed.

If we map these directly to screen pixels, the user's hand must physically touch the absolute edges of the camera's Field of View to reach the corners of their monitor.

2. The Ergonomic Failure

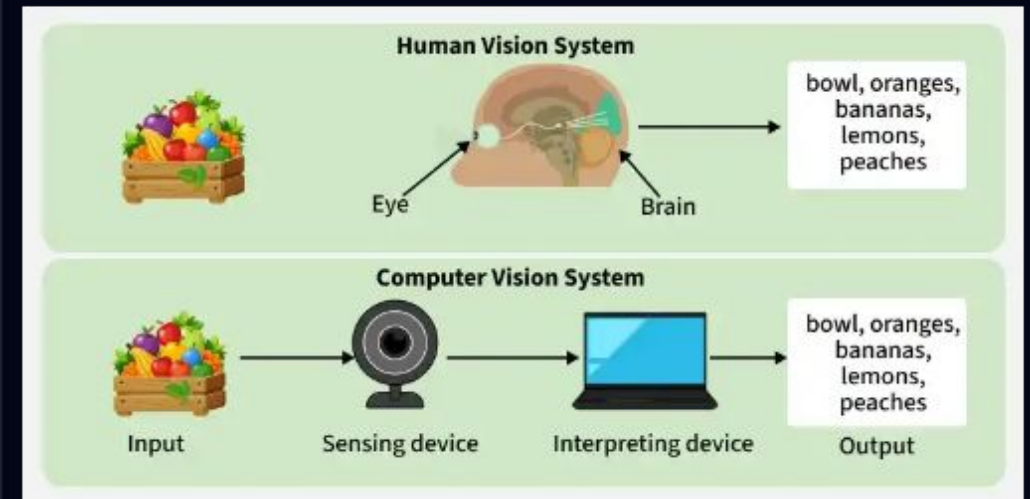
Human wrists are not designed to comfortably sweep the entire width of a webcam's sensor continuously.

Without adjustment, basic OS tasks like clicking a window's "Close" button become physically strenuous and impossible to perform reliably.

Defining the Active Zone

To solve the reachability issue, we define an inner bounding box within the raw camera frame.

- » We strip away an outer margin called `frame_reduction` (set to 100 pixels).
- » This creates a smaller, centralized interactable ^{area}
- » Any hand movement inside this reduced box is mathematically stretched to cover 100% of the display resolution.

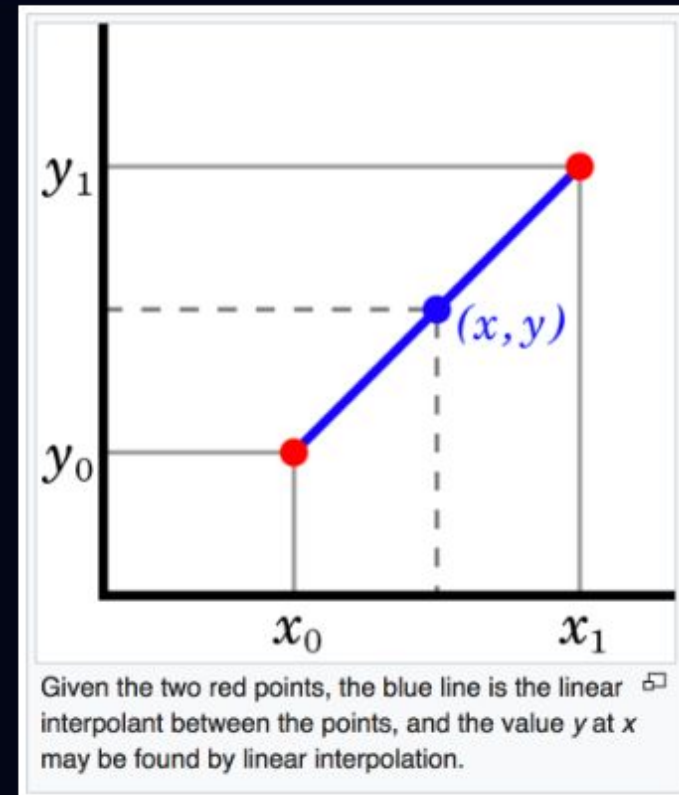


Linear Interpolation

We use NumPy's `interp()` function to map the finger's raw coordinates from the Active Zone to the Screen Resolution.

$$y = y_0 + (x - x_0) \frac{y_1 - y_0}{x_1 - x_0}$$

This translates the percentage of distance traveled inside the smaller box directly to the exact pixel coordinate on a 1920×1080 screen.



The Jitter Problem

High-Frequency Noise

Once we can reach the corners, a second issue emerges: Jitter.

Computer vision models have a margin of error. Even if your hand is perfectly still, the raw coordinate data rapidly fluctuates frame-by-frame. If we map the mouse cursor directly to this noisy data, clicking small desktop icons becomes impossible.



Exponential Moving Average

We solve the jitter by implementing an **Exponential Moving Average (EMA)**, which acts as a low-pass filter on the coordinate data.

- » Instead of snapping instantly to the new target, the cursor moves a fraction of the distance toward it.
- » By adjusting the smoothing variable ($k=5$), we create a weighted blend.
- » The cursor position is mostly anchored by where it already was, pulled gently toward the new raw target.

$$P_{\text{new}} = P_{\text{old}} + \frac{P_{\text{target}} - P_{\text{old}}}{k}$$

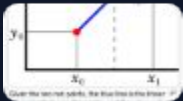
This mathematical damping absorbs high-frequency camera noise while perfectly preserving intentional hand swipes.

Image Sources



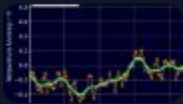
https://media.geeksforgeeks.org/wp-content/uploads/20260302172300663510/human_vision_system.webp

Source: www.geeksforgeeks.org



<https://dobrian.github.io/cmp/topics/linear-mapping-and-interpolation/linearmapping.png>

Source: dobrian.github.io



https://towardsdatascience.com/wp-content/uploads/2023/10/1cyE_dgxyzNezf9o1cRrmPA.png

Source: towardsdatascience.com

Relative Scrolling



Activation

Scrolling triggers when all 4 fingers are raised. The system dynamically locks the current Y position as the origin.



Reference Anchor

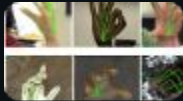
By capturing `scroll_ref_y` on entry, scrolling becomes position-agnostic. It doesn't matter where your hand is in the frame.



Execution

Moving up from the anchor origin scrolls up; moving down scrolls down, making the UX feel physically connected.

Image Sources



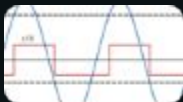
https://editor.analyticsvidhya.com/uploads/51525hand_crops.png

Source: www.analyticsvidhya.com



https://www.mdpi.com/technologies/technologies-14-00334/article_deploy/html/images/technologies-14-00334-g008-550.jpg

Source: www.mdpi.com



https://www.typhoon-hil.com/documentation/typhoon-hil-software-manual/Images/hysteresis_function_03.png

Source: www.typhoon-hil.com